

Autonomous Drone Racing – From MPCC to MPCCC

1st Niclas Schenk
Technical University of Munich
Munich, Germany
niclas.schenk@tum.de

1st Maximilian Christof
Technical University of Munich
Munich, Germany
m.christof@tum.de

Abstract—This paper addresses the challenge of autonomous drone racing in dynamic environments by developing and comparing two Model Predictive Contouring Control (MPCC) strategies. The first approach is a baseline MPCC that maximizes progress through an explicit reward term in its cost function, promoting aggressive flight. The second, which we term Model Predictive Contouring Curvature Control (MPCCC), removes the progress reward and instead adapts the reference speed based on upcoming trajectory curvature to enhance robustness. Extensive simulations in a highly randomized environment demonstrate a clear trade-off: the baseline MPCC is 19% faster but has a 51% success rate, whereas the MPCCC achieves a 75% success rate at the cost of slower lap times. One important finding is the controllers’ sensitivity to mass model mismatch, which is a primary cause of crashes. Both controllers successfully transfer to a real-world platform, where the MPCCC maintains its robustness, confirming the effectiveness of curvature-based speed adaptation for navigating unpredictable courses.

I. INTRODUCTION

Autonomous drone racing represents a formidable challenge, requiring navigation through complex, three-dimensional tracks at the physical limits of the quadrotor’s dynamics. The importance of this research lies in its potential to drive innovation in high-speed autonomous navigation. This work develops controllers for a highly dynamic simulation environment designed to bridge the sim-to-real gap, featuring significant randomizations in drone physical properties, environmental disturbances, and real-time positional updates of track objects.

To address these challenges, Model Predictive Control (MPC) is a suitable framework due to its ability to handle system constraints and optimize over a predictive horizon. However, traditional MPC struggles with the generation of offline trajectories, which are computationally expensive and inflexible. Model Predictive Contouring Control (MPCC) provides a more robust alternative by decoupling the spatial path from its time parametrization, allowing the controller to solve for the optimal speed profile online [1], making it well-suited for the unpredictable nature of drone racing.

Building upon this MPCC foundation, this paper develops and compares two distinct strategies that balance speed and safety. The first is a baseline MPCC that incorporates a progress reward to incentivize aggressive flight. The second, which we introduce as Model Predictive Contouring Curvature Control (MPCCC), takes a more conservative approach by

removing the progress reward and instead modulating the reference speed based on upcoming trajectory curvature. The central hypothesis is that this preemptive speed reduction will improve reliability at the cost of raw speed. The main contributions of this work are the design and implementation of this novel MPCCC, a comprehensive comparative analysis of the trade-offs between the two approaches in simulation and the real world, and an ablation study identifying model mismatch in mass as a critical factor for controller failure.

The remainder of this paper is structured as follows. Section II details the problem setup and the randomized simulation environment. Section III presents our methodology, covering trajectory generation and the implementation of our two controller approaches. Section IV analyzes the experimental results from both simulation and real-world flights. Finally, Section V concludes the paper and discusses future work.

II. THE CHALLENGE SETUP

A. Controller and Environment Setup

The high-level controller receives the observation

$$\mathbf{s} = [\mathbf{p}, \dot{\mathbf{p}}, \boldsymbol{\xi}, \dot{\boldsymbol{\xi}}, \mathbf{G}, \mathbf{O}],$$

with drone position $\mathbf{p} = [p_x, p_y, p_z]^\top$, velocity $\dot{\mathbf{p}}$, Euler angles, the drones roll, pitch and yaw attitude $\boldsymbol{\xi} = [\phi, \theta, \psi]^\top$ and their rates $\dot{\boldsymbol{\xi}}$. The gate and obstacle arrays $\mathbf{G}$ and $\mathbf{O}$ provide updated positional information and Euler angles for gates, along with visit status information for all gates and obstacles.

The drone can be controlled through two hierarchical control modes, since direct motor thrust control is infeasible due to latency issues in real-world deployment. The first mode uses high-level position control, where the controller’s output is action $\mathbf{a} = [x_{des}, y_{des}, z_{des}, \psi_{des}]$, with desired coordinates and yaw angle, while the second employs lower-level attitude control via command derivatives $\mathbf{a} = [\dot{f}_{c,cmd}, \dot{r}_{cmd}, \dot{p}_{cmd}, \dot{y}_{cmd}]^\top$, representing the derivatives of collective thrust and attitude references. The onboard attitude and motor controllers subsequently translate these high-level commands into precise motor inputs.

B. Environment and Physical Properties Randomization

To narrow the sim-to-real gap, the simulation environment is randomized by the following parameters at the hardest simulation level.

Drone Physical Properties: The drone’s mass is randomized with uniform distribution within ± 0.01 kg of the nominal value, while the inertia tensor components are disturbed by $\pm 1 \times 10^{-6}$ kg·m² along each principal axis. The initial drone position is randomized with position offsets of ± 0.1 m in x and y, $+0.02$ m in z, and orientation variations of ± 0.1 rad for all Euler angles.

Environmental Disturbances: The controller’s action value disturbances follow a normal distribution with standard deviation of 0.001, i.e., simulating sensor noise and actuator imperfections. Dynamics disturbances are uniformly distributed within ± 0.1 N across all three spatial dimensions, representing external forces such as wind gusts or ground effects.

Track Geometry Variations: Gate positions are randomized uniformly within ± 0.15 m horizontally and ± 0.1 m vertically, while gate orientations vary by ± 0.1 rad in yaw only. Obstacle positions experience similar horizontal variations of ± 0.15 m and vertical variations of ± 0.05 m, testing controller adaptability and collision avoidance capabilities.

III. MODEL PREDICTIVE CONTOURING CONTROL FOR DRONE RACING

Similar to the state dynamics in [2], our baseline MPC implementation for drone racing uses a 14-state quadrotor dynamics

$$\mathbf{x} = [p_x, p_y, p_z, v_x, v_y, v_z, \phi, \theta, \psi, \dot{\phi}, \dot{\theta}, \dot{\psi}, f_c, f_{c,\text{cmd}}, r_{\text{cmd}}, p_{\text{cmd}}, y_{\text{cmd}}]^T,$$

to generate attitude command derivatives $\mathbf{u} = [\dot{f}_{c,\text{cmd}}, \dot{r}_{\text{cmd}}, \dot{p}_{\text{cmd}}, \dot{y}_{\text{cmd}}]^T$ that track pre-computed time-optimal trajectories, but suffers from computationally expensive offline trajectory generation that can take minutes to hours. In contrast, Model Predictive Contouring Control (MPCC) reformulates the control problem by minimizing the distance to a spatial path while maximizing progress along it, requiring only a continuously differentiable 3D path parameterized by arc length rather than time [1]. MPCC introduces virtual progress dynamics with progress velocity v_θ as a control input, allowing the controller to solve the time allocation problem online and determine optimal sampling times along the path in real-time. The dual optimization objective balances contour error e_c^k minimization and progress maximization through the cost function $J = \sum_{k=0}^N q_c (e_c^k)^2 - \rho \theta_N$, encouraging full actuator utilization by dynamically adjusting sampling strategies to fully exploit available thrust and potential sub-optimality in the trajectory, while maintaining safety. The parameters q_c and ρ are used to balance these optimization objectives. This approach eliminates the need for conservative pre-computed trajectories and enables real-time adaptability to disturbances, making the MPCC well-suited for aggressive drone racing applications.

A. System Overview and Development Approach

The presented autonomous drone racing system comprises trajectory generation with path adaptation and Model Predictive Contouring Control for trajectory following. Development

follows an incremental approach, building complexity step-by-step to ensure gradual performance improvements.

Starting from a baseline Model Predictive Controller tracking pre-computed time-optimal trajectories, our implementation evolved to Model Predictive Contouring Control by implementing contouring and lag error calculations, followed by integration of a progress reward mechanism through virtual progress velocity v_θ . To address environmental uncertainties from gate and obstacle randomization, initial exploration of hard constraint formulations proved unreliable for gate avoidance. Therefore, in this work, we adopted an alternative approach inspired by [1], implementing dynamic contouring weight adaptation that increases penalties near gates while allowing path optimization freedom in open sections. Building on that, we developed another approach for speed control, where we calculate our targeted v_θ based on the upcoming curvature of the trajectory.

Finally, an update logic to modify the reference path when gate and obstacle positions deviate from nominal values, reducing collision risk, completes our system. The simulation environments visualization served as the primary verification tool throughout development.

B. Initial Trajectory Generation

Trajectory planning naturally splits into two complementary tasks: path geometry, which defines the spatial layout of the trajectory, and time parametrization, which determines how fast the reference point moves along that path. To handle the geometry, we fit a cubic spline through a small set of initial waypoints, producing a smooth, continuous trajectory.

$$x = \text{CS}_x(\theta), \quad y = \text{CS}_y(\theta), \quad z = \text{CS}_z(\theta),$$

where here θ parameterizes the progress along the path. Even if θ advances at a constant rate $\dot{\theta}$, this does not correspond to a constant real-world speed in m/s. In tight curves, where the parameter density $\|\nabla p(\theta)\|$ is high, the resulting speed is lower, whereas on straight segments with lower density a higher real-world speed is obtained.

A more advanced offline planner furthermore optimizes the snap $\ddot{p}(t)$ by

$$J = \int_0^T \|\ddot{p}(t)\|^2 dt,$$

where both the polynomial coefficients and the segment durations are variable [3]. This method produces very smooth flight trajectories but requires significantly more complex offline computations and fast online re-planning to handle dynamic gate shifts.

However, minimum snap trajectory generation proves incompatible with the dynamic environment requirements of this implementation for several reasons. First, minimum snap trajectories consist of piecewise polynomial segments with optimized transition times, making partial trajectory updates challenging when gate positions change [4]. Updating individual segments risks breaking the C^3 continuity requirements at segment boundaries, potentially violating the differentiability

constraints required for MPCC path tracking. Second, the temporal optimization inherent in minimum snap becomes redundant in MPCC formulations, since the virtual progress dynamics v_θ handle time allocation online [1]. The additional computational overhead of solving complex quadratic programs with equality and inequality constraints therefore, provides no performance benefit when the resulting time parameterization is discarded.

In contrast, cubic spline-based trajectory generation maintains C^2 continuity while enabling efficient local updates when environmental constraints change. This approach provides sufficient smoothness for aggressive drone maneuvers while preserving the computational efficiency and real-time adaptability essential for dynamic racing environments.

C. Trajectory Adaptation

The reference trajectory generation employs N waypoints $\mathbf{W} = \{w_0, w_1, \dots, w_N\}$ where $w_i \in \mathbb{R}^3$ to define the complete racing course trajectory. When environmental objects undergo position updates, our trajectory adaptation identifies waypoints within proximity of affected gates or obstacles and applies corresponding transformations.

The adaptation strategy baseline implements direct translation, where waypoints w_i are displaced by the same vector as their associated gate, or scaled displacement, where the displacement can be scaled by a factor relative to the gate position changes. However, these transformation methods can introduce trajectory artifacts, including excessively tight turns or closed loops, potentially causing tracking failures and performance degradation.

To mitigate these issues, our enhanced trajectory adaptation approach uses another base trajectory, which reduces waypoint density by removing redundant points and replacing waypoints near gates with gate-orthogonal waypoints. These new orthogonal waypoints shall help to maintain better trajectory smoothness while enabling early adjustment to gate orientation changes, particularly yaw variations.

Another trajectory adaptation we explored featured flying different trajectory segments around critical objects based on their updated positions, essentially creating bypass routes for specific obstacles. However, this approach was unsuccessful because the sharp trajectory transitions required to reconnect these segments often caused the controller to lose stability.

D. Visualization during Development

The simulation environment employs CrazyFlow for real-time visualization and development support [5]. This visualization framework proved to be an important tool for validating controller implementations and identifying behavioral discrepancies during development phases. It additionally helped with debugging of new controller features, enhancing our understanding, and accelerating the development progress.

During development, we implemented multiple components in the visualization for verification: the predicted trajectory over the MPC prediction horizon, the actual flown path, the complete planned trajectory for the entire racing course, and

the current target progress parameter θ . Additional visualizations include contouring and lag errors, and the dynamic gate weight visualization, highlighting trajectory adherence prioritization.

This real-time feedback aids rapid identification of implementation deviations and debugging of controller components, significantly accelerating the development cycle compared to relying solely on numerical performance metrics.

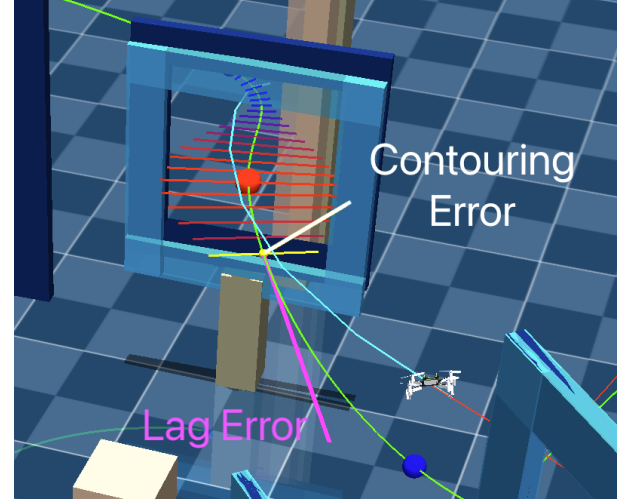


Fig. 1: Simulation environment visualization showing predicted trajectory, actual flight path, contouring errors, and dynamic Gaussian weight adaptation near racing gates.

E. Approach 1: MPCC with Dynamic Contouring Weights and Progress Penalization

1) *Trajectory Parametrization*: The cubic spline trajectory generation inherently provides beneficial speed characteristics without additional parametrization requirements. While cubic splines consist of discrete points forming a continuous trajectory, their adaptive point density—higher in curved regions—causes the progress parameter θ to naturally encounter more waypoints in corners than on straight segments.

This parametrization characteristic means that even with constant virtual progress speed v_θ , the effective reference point advancement $\dot{\theta}$ automatically decreases in curved trajectory sections due to the increased point density. Rather than viewing this as a limitation, this inherent behavior provides a desirable speed modulation effect that complements the explicit curvature-based adaptation.

The automatic deceleration in corners through trajectory parametrization enables more aggressive controller tuning for straight segments, as the system benefits from both explicit curvature control and implicit geometric speed reduction without additional computational overhead.

2) *Implementation of Contouring and Lag Errors*: The contouring error measures the lateral deviation from the desired trajectory, and the lag error quantifies the longitudinal delay along the path (see Figure 1). Since the exact determination of θ^* constitutes a nonlinear optimization problem, only these

errors are estimated in the following. The detailed definitions can be found in [1].

3) *Implementation of Dynamic contouring weights:* The dynamic contouring weight adaptation modulates the penalty coefficient q_c in the MPCC cost function based on proximity to gates, creating *Gaussian-shaped emphasis regions* that enhance tracking precision in critical areas while maintaining computational efficiency. The increase in contouring weight before gates, once it is implemented, is also visualized in 1, where larger bars indicate larger contouring weights.

The implementation identifies reference progress values $\theta_{gate,i}$ corresponding to each gate center. For any progress parameter θ within the prediction horizon, the contouring weight evaluates the influence of all gates through:

$$q_c(\theta) = \max \left(w_{base}, \max_i \left[w_{base} + (w_{peak,i} - w_{base}) e^{-\frac{(\theta - \theta_{gate,i})^2}{2\sigma_i^2}} \right] \right)$$

where w_{base} represents the baseline contouring weight, $w_{peak,i}$ the peak weight for gate i , and σ_i controls the spatial extent of influence around each gate.

This formulation ensures $q_c(\theta) \geq w_{base}$ throughout the trajectory, with significant increases near gates. The elevated penalties reduce trajectory deviation risks near critical regions, while smooth Gaussian transitions prevent abrupt control changes. The maximum operation across gate influences prevents additive penalty accumulation in overlapping regions, maintaining numerical stability while preserving safety characteristics.

When we put all that together the controller's policy $\pi(\mathbf{x})$ is determined by:

$$\pi(\mathbf{x}) = \underset{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}}{\operatorname{argmin}} \sum_{k=0}^{N-1} L(\mathbf{x}_k, \mathbf{u}_k)$$

The stage cost L at each step k balances path tracking accuracy, progress maximization, and control input regularization:

$$L(\mathbf{x}_k, \mathbf{u}_k) = q_c(\theta_k)(e_k^c)^2 + q_l(e_k^l)^2 - \mu v_{\theta,k} + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k$$

where μ is the weight for the virtual progress velocity $v_{\theta,k}$. The quadratic control penalty $\mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k$ uses diagonal weighting matrix $\mathbf{R}$ to regularize the control vector $\mathbf{u}_k = [\dot{f}_{c,cmd,k}, \dot{\phi}_{cmd,k}, \dot{\theta}_{cmd,k}, \dot{\psi}_{cmd,k}, \dot{v}_{\theta,k}]^\top$, where the derivatives represent the collective thrust command and Euler angle command rates, along with the derivative of the progress parameter v_{θ} , ensuring smooth flight characteristics and actuator constraint satisfaction.

F. Approach 2: Curvature-Based Speed Adaptation

The approach begins with a reference trajectory and implements curvature-based deceleration for tight curves that in our case primarily result from trajectory updates due to moved obstacles or gates. This preemptive speed reduction enables more precise tracking of dynamically updated trajectories, improving controller reliability while minimizing

time penalties compared to standard MPCC approaches that lack curvature adaptation, which we name Model Predictive Contouring Curvature Control (MPCCC).

1) *Curvature Estimation:* The curvature-based speed adaptation modulates the virtual progress parameter θ according to local trajectory curvature, enabling the drone to decelerate smoothly in tight turns while maintaining high speeds on straight segments. This approach improves tracking robustness without significant time penalties.

Starting from a cubic spline reference trajectory through predefined waypoints, the algorithm computes first and second derivatives:

$$\begin{aligned} \mathbf{v}(\theta) &= (\dot{x}, \dot{y}, \dot{z}) = (CS'_x(\theta), CS'_y(\theta), CS'_z(\theta)), \\ \mathbf{a}(\theta) &= (\ddot{x}, \ddot{y}, \ddot{z}) = (CS''_x(\theta), CS''_y(\theta), CS''_z(\theta)). \end{aligned}$$

The curvature $\kappa(\theta)$ follows from the standard geometric formula:

$$\kappa(\theta) = \frac{\|\mathbf{v}(\theta) \times \mathbf{a}(\theta)\|}{\|\mathbf{v}(\theta)\|^3 + \varepsilon},$$

where ε prevents division by zero. The parameter speed adaptation then implements:

$$v_{\theta} = \frac{v_{\theta,base}}{1 + \alpha_{\kappa} \kappa(\theta)},$$

where $v_{\theta,base}$ represents the nominal virtual advance speed and α_{κ} controls the sensitivity of curvature-based deceleration. This formulation automatically reduces speed in regions with high curvature while enabling high speeds on straights. This enhances the inherent slowdown caused by tighter parameterization in sections of the cubic spline trajectory with greater curvature.

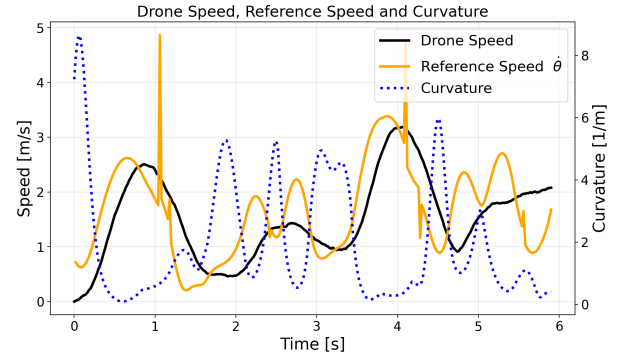


Fig. 2: Reference parameter speed $\dot{\theta}_0$ (red) and actual drone speed (orange) current curvature at reference parameter θ_0

Figure 2 demonstrates the expected behavior where increased trajectory curvature results in reduced v_{θ} virtual speed, with the actual drone speed correspondingly decreasing when the v_{θ} reduction persists for sufficient duration. The observed temporal lag between v_{θ} and drone speed aligns with the MPCC formulation, where the drone intentionally trails the reference progress parameter θ by the lag error e_l .

The visualization confirms that both v_{θ} and drone speed exhibit significant deceleration in high-curvature regions, providing the controller with enhanced reaction time for unforeseen

disturbances and improved handling of tight corners, thereby increasing overall system robustness.

These changes result in an updated controller policy, which reads:

$$\pi(\mathbf{x}) = \underset{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}}{\operatorname{argmin}} \sum_{k=0}^{N-1} L(\mathbf{x}_k, \mathbf{u}_k)$$

The stage cost L is accordingly modified to remove the explicit progress maximization term, focusing purely on tracking accuracy and input regularization:

$$L(\mathbf{x}_k, \mathbf{u}_k) = q_c(\theta_k)(e_k^c)^2 + q_l(e_k^l)^2 + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k$$

The regularized control vector

$$\mathbf{u}_k = [\dot{f}_{c,\text{cmd},k}, \dot{\phi}_{\text{cmd},k}, \dot{\theta}_{\text{cmd},k}, \dot{\psi}_{\text{cmd},k}]^\top$$

no longer includes $\dot{v}_{\theta,k}$, as the virtual progress velocity is not part of the optimization anymore.

a) *Parameter Adaptation:* While the system does not actively compensate for mass variations, more conservative controller parameters enhance stability under varying drone masses. We increase the state constraints for roll ϕ and pitch θ , and also introduce one for v_θ to ensure the drone remains in stable flight condition and does not get too fast so it can always react to sudden object changes on the course. The extensive mass randomization in simulation (± 0.01 kg) exceeds typical real-world variations, making active mass adaptation a lower priority since actual flight conditions exhibit significantly smaller parameter deviations.

Both MPC and MPCC approaches underwent comprehensive parameter evaluation through systematic parameter sweeps. The tuning process evaluated both contouring base weights and dynamic gate peak weights, prediction horizon steps, time parameters for base v_θ calculation, and the curvature sensitivity parameter α_κ to optimize deceleration behavior in corners. The resulting retrieved parameters helped us to improve controller stability across the anticipated operating conditions.

IV. RESULTS AND COMPARISON

A. Experimental Setup: Simulation

The simulation framework utilizes *Gymnasium* as the primary RL environment interface with *Google Fire* for command-line functionality, while core drone racing components are implemented as custom modules within the `lsy_drone_racing` package, representing a hybrid approach combining established frameworks with domain-specific homebrew solutions.

During development, we worked with random seeding (`int(time.time()) % 100000`) in order to avoid bias from scenario selections for the few runs that we conducted between small changes. However, this random seeding revealed significant performance variability, with successful completion rates fluctuating by up to 30% across identical controller configurations. To ensure reproducible comparisons

between optimization-based and RL controllers, a *fixed seed parameter* (`seed=config.env.seed`) was implemented, enabling both methods to encounter identical environmental conditions.

All evaluations were conducted using the *Level 2 configuration* over *100 simulation runs* to achieve statistical significance. Additionally, we implemented a datalogger that records completion times, gate passage counts, drone mass variations, and success rates, which constitute the primary metrics used for evaluation.

B. Performance Comparison MPCC vs. MPCCC in Simulation

TABLE I: Comparison of simulated performance for MPCC and MPCCC.

Controller	Mean	Var.	Best	Worst	Pass Rate
MPCC	4.983	0.0302	4.880	6.180	51%
MPCCC	5.929	0.0015	5.800	6.000	75%

A direct comparison of the baseline Model Predictive Contouring Control (MPCC) and the proposed Model Predictive Contouring Curvature Control (MPCCC) reveals a distinct trade-off between speed and robustness, as detailed in Table I.

The baseline MPCC achieves a faster mean lap time of 4.983s, compared to the MPCCC's 5.929s. This speed advantage, however, corresponds with lower reliability, as the MPCC's aggressive nature makes it less capable of adapting to dynamic gate and obstacle updates, resulting in a pass rate of 51%. In contrast, the MPCCC achieves a 75% pass rate. The curvature-aware speed adaptation of the MPCCC effectively prevents crashes in tight corners, which validates the core concept of the approach, albeit with a clear penalty in overall lap time.

Furthermore, the variance in completion times highlights the different stability characteristics of the two controllers. The MPCC exhibits a much higher variance of 0.0302 than the MPCCC's 0.0015. This can be attributed to the MPCC's tendency to operate on the edge of its stable flight envelope. It achieves very fast times under favorable conditions but slows down considerably when forced to recover from near-unstable states. The simulation results show that the MPCC's proactive speed management in high-curvature sections avoids these borderline states entirely, as it was designed to do, leading to more consistent and predictable performance.

Unfortunately, we were unable to tune the parameters of the MPCCC for the improved trajectory. Consequently, the above results are from the curvature-based MPCCC are for the following on our updated base trajectory. We anticipate that the performance of the MPCCC could be further enhanced beyond the current version by adapting its parameters to the characteristics of the smoother trajectory.

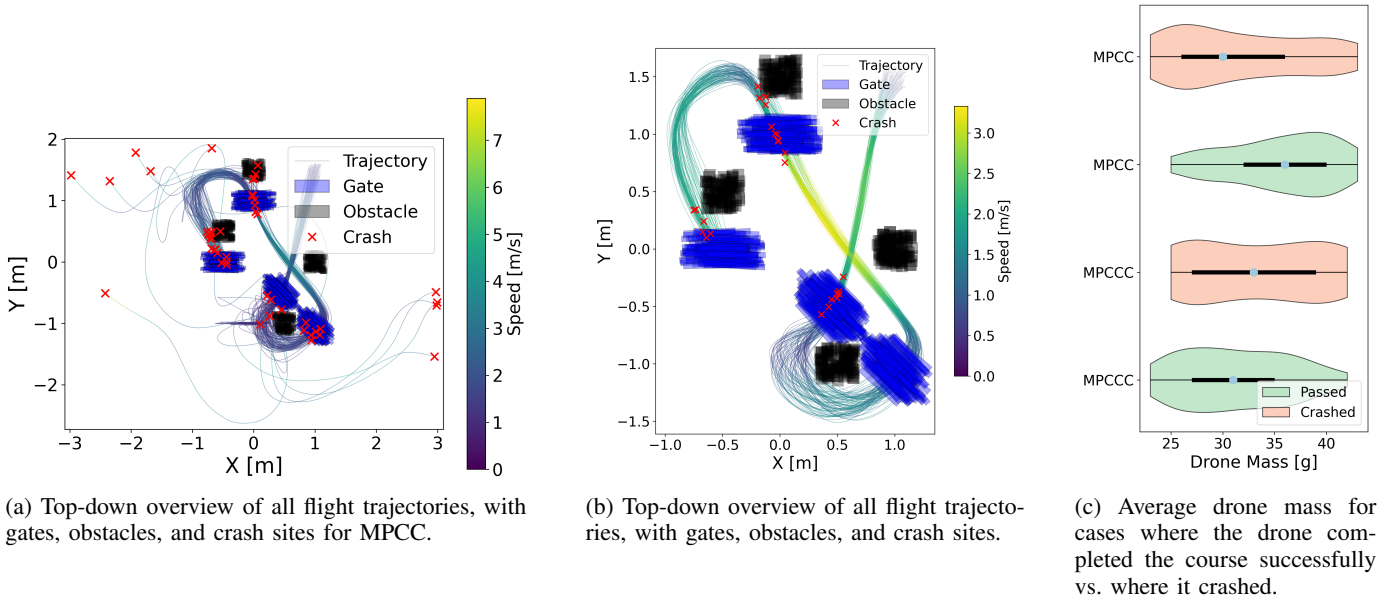


Fig. 3: Performance comparison between MPCC and MPCCC

C. Ablation Studies: Environment Randomizations

This subsection evaluates the influence of the previously introduced parameter optimizations across different levels of environmental randomization. Our simulations over 100 runs show that both controller variants can master the simulation's Level 0, which includes no randomizations with a 100% success rate. On Level 1, which introduces all randomizations (including mass randomization) except for track object updates, the MPCCC achieves a 100% success rate, while the MPCC scores only 91%. The MPCC's higher sensitivity is an expected consequence of its more aggressive character, which can lead to unrecoverable states more often than for the MPCCC, which slows more proactively. Nonetheless, both controllers show excellent resistance to these disturbances, even though they were not specifically developed to handle them.

However, the low pass rates presented in Table I for Level 2 reveal that reacting to dynamic gate and obstacle updates has a significant impact on performance. This challenge particularly affects the baseline MPCC, as it operates closer to the physical limits of the drone, leaving less margin for error when reacting to sudden changes. The top-down flight visualizations in Figure 3a and 3b confirm this, showing that the MPCC most frequently crashes into gates and obstacles when it is unable to react to trajectory updates in time.

Furthermore, evaluating the correlation between drone mass and crash rates on Level 2 reveals another critical factor. As shown in Figure 3c, the MPCC crash more often when the drone's mass is lower than its nominal value. While the MPCCC shows greater robustness across the entire mass spectrum due to its conservative slowdowns in tight corners. This behavior most likely stems from the model mismatch: when the drone is lighter than the controller anticipates, the

same control inputs produce unexpectedly high accelerations. As a consequence, the drone is "faster than it thinks," causing it to lose track of the trajectory in curves or rendering it unable to react to updated object positions, ultimately leading to a collision.

D. Experimental Setup: Real-World

The real-world experimental setup utilized *Vicon motion capture systems* for precise drone localization, with provided estimators handling position estimation under varying hardware conditions. Unlike simulation's controlled randomizations, real-world variability is manifested through hardware-specific factors, including motor thrust variations, propeller damage, aerodynamic drag changes from Vicon tracking ball configurations, and potential drone component damage or missing parts. Variable takeoff positions further contributed to experimental uncertainty.

All real-world evaluations consisted of *10 runs per controller* with randomized gate and obstacle positions within allowable ranges. The reduced run count reflects practical constraints of physical hardware testing in the lab while maintaining sufficient data for comparative controller assessment.

E. Comparison of Simulation and Real-World Performance

As illustrated in Fig. 3c, the drone mass had a pronounced influence in simulation: with our deliberately aggressive tuning, even small mass deviations led to markedly different success rates. In the real world, however, the vehicle mass was essentially constant at about 33 g. Consequently, the controller could fly noticeably faster on the real platform than in simulation, where additional randomizations (e.g., gate and obstacle positions, mass randomizations) had to be accounted for.

Despite these randomizations during training, the controller exhibited a robust yet aggressive flight style on the physical track. It completed the course reliably and in short times (see Table II). The fact that such an aggressive policy remained stable in reality underlines the soundness of our MPCC implementation.

TABLE II: Real-world controller performance over 10 runs. MPCC* denotes the MPCC variant *without* online updates of track objects.

Controller	Mean Time [s]	Variance [s ²]	Best Time [s]
MPCC*	4.46	0.0004	4.42
MPCCC	5.24	0.0035	5.14
MPCC	5.35	0.0049	5.24

V. CONCLUSION

This work presented and analyzed two Model Predictive Contouring Control strategies, highlighting the inherent trade-off between speed and robustness in dynamic drone racing environments. The results show a clear distinction: the baseline MPCC is faster but less reliable (51% pass rate), as its aggressiveness makes it vulnerable to sudden trajectory changes. In contrast, the curvature-aware MPCCC is more robust (75% pass rate) but slower, validating that its proactive speed reduction is effective but introduces a time penalty. Key limitations identified for both controllers include a sensitivity to mass model mismatch, which causes instability when the drone is lighter than expected, and the generation of overly sharp turns from the current trajectory adaptation method.

A. Future Work

Future work could focus on creating a hybrid controller that merges the two presented approaches, using curvature to regulate the maximum virtual velocity to potentially combine the speed of the MPCC with the robustness of the MPCCC. To address failures originating from sharp trajectory updates, the current cubic splines could be replaced with B-splines to generate smoother and more reliable paths during dynamic updates [6]. Finally, to counter the critical issue of model mismatch, a simple online adaptation mechanism could infer mass deviations from initial flight behavior and adjust thrust constraints accordingly, significantly improving robustness to this key randomization.

REFERENCES

- [1] A. Romero, R. Penicka, and D. Scaramuzza, “Time-Optimal Online Replanning for Agile Quadrotor Flight,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7730–7737, Jul. 2022, ISSN: 2377-3766. DOI: 10.1109/LRA.2022.3185772. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9805699> (visited on 05/21/2025).
- [2] B. Akbari, J. Frank, and M. Greeff, *Tiny Learning-Based MPC for Multirotors: Solver-Aware Learning for Efficient Embedded Predictive Control*, arXiv:2410.23634 [cs], Nov. 2024. DOI: 10.48550/arXiv.2410.23634. [Online]. Available: <http://arxiv.org/abs/2410.23634> (visited on 07/22/2025).
- [3] C. Qian, R. Chen, P. Shen, Y. Fang, J. Yan, and T. Li, *Minimum Snap Trajectory Generation and Control for an Under-actuated Flapping Wing Aerial Vehicle*, arXiv:2311.01003 [eess], Nov. 2023. DOI: 10.48550/arXiv.2311.01003. [Online]. Available: <http://arxiv.org/abs/2311.01003> (visited on 07/22/2025).
- [4] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *2011 IEEE International Conference on Robotics and Automation*, ISSN: 1050-4729, May 2011, pp. 2520–2525. DOI: 10.1109/ICRA.2011.5980409. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/5980409> (visited on 07/22/2025).
- [5] *Crazyflow: Simulation and visualization framework*, <https://github.com/utiasDSL/crazyflow>, Accessed: 20.06.2025.
- [6] L. Laban, M. Wzorek, P. Rudol, and T. Persson, *Custom Non-Linear Model Predictive Control for Obstacle Avoidance in Indoor and Outdoor Environments*, arXiv:2410.02732 [cs], Oct. 2024. DOI: 10.48550/arXiv.2410.02732. [Online]. Available: <http://arxiv.org/abs/2410.02732> (visited on 07/22/2025).